

Spec Enforcement Against a Running System

Given a spec that passed a deterministic gate, how do you mechanically prove the running system satisfies it? Findings across contract testing, browser automation, runtime verification and evidence recording; a recommended conformance harness; what it cannot verify; and the effort to build it.

1. The headline findings

First: the thing you want mostly does not exist as a product, but every piece of it exists as a tool. There is no off-the-shelf system that takes a spec and mechanically proves a running deployment satisfies it. Specifically, no tool anywhere diffs "controls declared in a spec" against "controls found on the live page" — broken-*link* checking is a solved commodity; broken-*button* checking is not a product category. But the primitives — real-browser automation with protocol-level network capture, schema-driven API verification against a live URL, request-ID correlation into server logs, signed evidence bundles — are all mature, mostly free, and composable. The custom part is small and it is exactly your spec format's shape: your gate already forces every control to declare its endpoint, which means **the spec is the manifest** the missing tool needs.

Second: the industry's flagship "contract testing" is explicitly not this. Pact's own documentation recommends *against* verifying a deployed provider: "Verifying pacts against an already deployed provider will mean you don't get the benefits that contract testing was intended to provide" — it is a CI-time technique requiring controllable provider states ([Pact docs](#)). Spring Cloud Contract is the same shape. Anyone selling "contract testing" as live-system proof is reselling a build-time tool.

Third: the category is littered with recent corpses, which tells you what to build on. Dredd (the classic spec-vs-live-server validator) was archived November 2024; Optic (spec-vs-traffic drift) was archived January 2026 after the Atlassian acquisition; Keptn (SLO quality gates) was archived by the CNCF in September 2025 — all verified against the repos and the CNCF page. The

survivors are boring and old: Playwright, Hypothesis/Schemathesis, plain schedulers, Sigstore. Build on those.

2. Answers to the six sub-questions

2.1 Which criteria can be verified fully automatically, and where is the line

The crispest available frame is Bach and Bolton's checking-versus-testing distinction — *checking* is "the mechanistic process of verifying propositions about the product" by algorithmic decision rules ([Testing and Checking Refined](#)) — combined with SRE practice's operational version: only what is expressible as a measurable indicator over observable signals can be gated ([Google SRE book, ch. 4](#)).

A criterion is machine-checkable against a live system if and only if it is a proposition over observable network, trace, or metric behaviour with an algorithmic decision rule. On the automatable side: response schemas, status codes, and error envelopes; state transitions visible through the API (create-then-get, pagination invariants, idempotency); enumerated authorisation cases (role X gets 403 on endpoint Y); timing and availability SLOs; span-level assertions on real traces; and no-5xx robustness under generated inputs (probabilistic — it falsifies, never proves). On the other side, fundamentally not automatable: aesthetic and judgement qualities (no decision rule exists); **universal absence claims** — "never leaks," "no vulnerabilities" — because finite observation cannot establish a universal (Dijkstra's dictum: testing shows presence of bugs, never absence); emergent fitness-for-purpose; long-horizon properties (durability over years); and any truth living outside the observable interface (was the email actually read; is the price commercially right).

There is an honest middle band the brief should plan for: properties automatable **only with the system's cooperation** — seeded test data, test tenants, controllable states. That is precisely why Pact refuses live providers. Your spec's `environment` and `externals` fields are where that cooperation gets declared.

2.2 What exists today — real versus marketing

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
Schema-driven API verification	Schemathesis (property-based, from OpenAPI, against any base URL)	Yes	Real – the best-evidenced tool in this entire space: peer-reviewed benchmark on 16 real services found 1.4×–4.5× more unique defects than the second-best of eight fuzzers (arXiv 2112.10328 ; author-written but peer-reviewed with a public benchmark; venue label soft-confirmed). Actively maintained, built on Hypothesis.
Stateful REST fuzzing	RESTler (Microsoft)	Yes	Real, research-grade; 28 bugs in GitLab in the ICSE 2019 paper; internal Azure claims not independently audited (repo)
Example/contract conformance	Microcks	Yes	Real – CNCF incubating May 2026; shallower than Schemathesis (examples and schemas, not generated properties)
Spec-example validation	Dredd	Yes	Dead – archived Nov 2024. Do not adopt.
Spec-vs-traffic drift	Optic	–	Dead – archived Jan

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
			2026. oasdiff is alive but spec-vs-spec only; live drift detection is an open gap.
Consumer-driven contracts	Pact, Spring Cloud Contract	No – by design	Real and mature, but CI-time; their own docs say not to point them at deployed providers
Property-based testing at the boundary	Hypothesis → Schemathesis	Yes	Real. UI-level temporal-logic testing (Quickstrom, PLDI 2022) is real research, honestly niche.
Trace-based assertions	Tracetest; plain SLO monitors	Yes	Real mechanism, churning category – Keptn archived Sep 2025; Tracetest's OSS momentum has visibly slowed
Runtime verification from formal specs	AWS's internal PObserve (validates production logs against P specs – ACM Queue); NASA's Copilot (avionics)	Yes, post hoc	Real but unavailable – no adoptable general-purpose product exists. This is the most instructive gap: the best team in the world at this built it in-house because they couldn't buy it.

CATEGORY	BEST-EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
Model checking (TLA+/P)	TLA+	No	Real – but it verifies the <i>design</i> , not the running code, and MongoDB's attempt to trace-check the running server against its TLA+ spec was abandoned as costing more than it was worth (VLD B 2020). Spec-derived <i>test generation</i> succeeded in the same study.
eBPF behavioural rules	Falco	Yes (host-level)	Real for security deny-lists; nobody credibly claims spec conformance with it
Synthetic monitoring (scheduled real-browser checks)	Checkly (Playwright-native), Datadog Synthetics	Yes	Real and battle-tested. Standalone OSS "synthetic monitoring platforms" are marketing-thin; scheduled CI running Playwright is the credible free route.
Browser automation with network evidence	Playwright	Yes	Real – protocol-level capture, HAR recording, trace archives, accessibility-tree snapshots, with honestly documented

CATEGORY	BEST - EVIDENCED TOOL	WORKS AGAINST A DEPLOYED URL?	VERDICT
			blind spots (\$2.3)

2.3 The cheapest reliable proof that every declared control reaches its declared endpoint, on the live page

Four layers, three of them free, one small custom piece:

The manifest is generated from the spec. Your gate's R5 already forces every control to carry `endpoint` or `display_only`, and every surface to carry its route. That is the manifest the missing tool needs: control (role + accessible name) → expected endpoint (method + pattern). Nothing to author twice; drift between spec and manifest is impossible because they are the same document.

The census. Playwright's accessibility-tree snapshot (`toMatchAriaSnapshot`) freezes the full set of roles, names, and states on the live page as reviewable YAML ([docs](#)). Diffing that against the spec's declared controls fails the run in either direction: a declared control missing from the page, or an undeclared control present. Known limit: it enumerates *accessible* controls — a clickable `div` with no role is invisible to it, which is itself a defect worth surfacing.

The click-proof. For each manifest row, on the production URL: a real click via `getByRole`, then block until the matching request/response is observed (`waitForResponse` with URL pattern and 2xx), with the context configured to block service workers (Playwright's own recommendation when interception matters — [network docs](#)), HAR recording on, and trace recording on. This is a mechanical click → endpoint proof with no mocking anywhere. Playwright's documented blind spots, planned around rather than discovered later: service-worker-served requests can be invisible or header-stripped in HAR (hence the block setting); `keepalive` / `sendBeacon` traffic is flaky across engines; WebSocket frames are not represented in HAR at all (the format was never standardised and has no frame entries); route handlers only see requests issued after registration.

The server-side confirmation — the step that closes tonight's failure class. A 2xx in the browser proves *something* answered; it does not prove the declared handler ran (a CDN or cache can answer). The pragmatic standard is request-ID injection: the harness stamps every request with a unique `X-Request-Id: verify-<run>-<control>` and then queries the backend's logs for it — the long-established correlation pattern ([Heroku](#)). The stronger version is minting a W3C `traceparent` header *from the harness* and

looking up the server span in the tracing backend; Datadog productised exactly this correlation for its synthetics ([docs](#)). Do not gate anything on in-page browser OpenTelemetry — its own maintainers describe it as "experimental and mostly unspecified"; header injection from the harness is the stable path.

Cost. Playwright is free. Scheduling via CI cron is free at daily granularity. Checkly's free tier covers 1,000 hosted browser-check runs a month, Starter is \$24/month (verified against their pricing page); Datadog is \$12 per thousand runs and only makes sense if already paying for Datadog, where it buys the click → server-span correlation as a product feature. The custom piece — spec → manifest → generated checks — is the only thing that has to be built.

Production safety, from documented practice (Microsoft's engineering playbook on synthetic tests against production with test users; Google SRE on probes that replay known-good requests): read-only controls are verified by real submission; mutating controls run in a test tenant with a cleanup contract, or carry a synthetic-traffic marker header the backend recognises; destructive controls without a test tenant are the one category that cannot be safely click-proven on production, and the spec's `reversible` / `rollback` fields are where that surfaces.

2.4 Verifying criteria that have no runnable check, without a model's opinion

Three findings. First, the population of such criteria should be small by construction — your gate's banned-word rule and `human: true` labelling already push everything checkable into runnable form, so what remains is genuinely judgement: appearance, tone, fitness. Second, **the smallest thing a human must look at is a diff, not a page**. Visual regression practice (Percy, Chromatic; or open-source pixel-diffing) reduces "does this still look right" to "approve these N changed regions against the last approved baseline" — the human judges deltas from a known-good state, not the whole surface each time. The same pattern generalises: for each `human: true` criterion, the record should present the *evidence pack* the spec's `evidence_required` field names (a specific screenshot, a specific response body) and nothing else. Third, where a model is used at all, the evidence from this project's earlier research is that it belongs *before* the human as a flag-raiser, never as the decider — LLM judges anchor on confident prose (AUROC ≤ 0.65 where a trivial environment-state classifier reached 0.83–0.95) and flip under pushback; a script routes, a human decides, a model may only annotate. The auditable unit is: criterion id → evidence pack → named human → dated verdict.

2.5 The false-success numbers

Two bodies of evidence, one for machine reporters and one for human ones, pointing the same way.

When the reporter is an LLM system that did the work: self-reported completion is false in 13–79% of failures depending on model; false success is 45–48% of failures in tool-use benchmark domains and 75.8% of self-assessing trajectories in AppWorld ([Confident Closing, 2026](#)); agent-written tests average ~25 print statements per 5 assertions, and prompting for more tests doesn't change outcomes ([2602.07900](#)); adding a same-family model as checker approves its own family's output preferentially ([Panickssery, NeurIPS 2024](#)). Independent mechanical verification cut false success to ~3% of failures in the one study that measured the intervention.

When the reporter is the human who did the work: developer self-testing removes roughly 20–50% of defects while independent formal inspection removes 60–97% (Capers Jones' defect-removal tables — consultancy data, ~15 years old, the most-cited and least-auditable numbers in the field); Fagan's original IBM study had inspections finding 82% of defects (1976, replicated directionally for decades); and developers measured as higher in confirmation bias write tests that *verify* rather than falsify and ship higher defect density ([Calikli & Bener 2013](#), five industrial datasets). The green-checkmark infrastructure itself lies at scale: ~16% of Google's 4.2M tests showed flakiness (Google testing blog, 2016, verified); 4.6% of test cases flaky at Microsoft, hitting 14–52% of builds ([ICSE-SEIP 2019](#)). One honest gap: the false-pass direction — tests that assert nothing and stay green — is essentially unquantified in the public literature.

The design consequence is the one your constraints already state: the verifier must not be the builder, and the numbers above are why that is not a preference but a measured necessity.

2.6 Recording results so "done" is auditable

The evidence bundle per verification run: the Playwright trace archive (per-action screenshots, DOM snapshots, console, network — self-contained), the HAR (scrubbed — it captures cookies and headers, and the format was never standardised, so treat it as secondary evidence), the manifest snapshot (which is the spec version), the request-ID or trace-ID lookup results from the backend, the commit SHA, target URL, and timestamp. JUnit XML is not evidence — it records that assertions passed, not what was observed, and is trivially forgeable.

For the audit chain: write-once storage (object-lock) clears the practical audit bar today — SOC 2 evidence collectors gather less (timestamped screenshots and API pulls). The upgrade path exists and is shipping piece by piece: the in-toto attestation spec has a stable

`test-result/v0.1` predicate designed exactly for "verify required tests ran and passed" (verified in the spec repo), GitHub's attest action signs arbitrary predicates over artifact digests with Sigstore, and Rekor provides the append-only log. Honest status: nobody is doing signed test attestations at scale yet — the predicate exists, adoption is demos — so this is an early-adopter position, consistent with the verification-record direction of the earlier reports, not established practice.

3. Recommended approach

Build a conformance harness that compiles the approved spec into three check suites plus an evidence bundle, run by a verifier identity that is not the builder.

Reasoning follows each part.

1. **Spec → manifest → generated UI census and click-proofs** (the custom piece; nothing on the market does it). Census by accessibility snapshot; per-control click with network wait; service workers blocked; request-ID injected; server log/trace lookup closes the loop to the handler. *Why*: this is the exact failure class you hit — panels that render with dead controls — and no existing category catches it: RUM dead-click detection is statistical and needs real users; synthetic monitors check journeys someone scripted, not the declared control inventory.
2. **Spec → API conformance suite** against the deployed base URL: the spec's `actions[]` endpoints and `acceptance[]` verify commands run as-is, plus Schemathesis over the OpenAPI description if one exists, for schema conformance and no-5xx robustness beyond the enumerated criteria. *Why*: it is the mature end of the market, the one tool with a peer-reviewed benchmark, and free.
3. **Evidence bundle + verdict record** per run, in write-once storage, with the in-toto test-result attestation as the signing layer when you want "done" to be verifiable offline. Human-judgement criteria get evidence packs and diff-based review; a model may pre-annotate, a script tallies, a human signs.
4. **The verifier is a separate identity**: its own repo, its own credentials, no write access to the system under test beyond the test tenant, and the builder cannot modify the harness or the manifest between approval and verification (the manifest is the approved spec, version-pinned). *Why*: every number in §2.5.
5. **Schedule it**. A pass proves a point in time; the same suite on a daily cron (free) or Checkly free tier is what makes "still done" a standing fact rather than a memory. The dead-tool history in §1 argues for owning the scheduler-agnostic suite and treating any hosted runner as replaceable.

What was considered and rejected: Pact-style CDC (build-time by design); TLA+/model checking as the verification layer (verifies the design, not the deployment; MongoDB abandoned trace-checking as not worth it — though spec-derived *test generation* is worth revisiting later); eBPF conformance (nobody credibly does it); in-page browser OTel (experimental by its maintainers' own description); LLM-as-verifier (the numbers in §2.5 are the whole argument, and your constraint already forbids it).

4. What the recommended approach cannot verify

Stated plainly. It cannot verify business-logic *correctness* of a handler — only that the declared handler ran and returned the declared shape; a handler that runs and computes the wrong answer passes unless an acceptance criterion pins the answer. It cannot see controls invisible to the accessibility tree (handler-less clickable divs — mitigated, not solved, by treating census mismatches as defects). It cannot reliably capture `keepalive` / `sendBeacon` traffic or WebSocket payloads in HAR evidence. It cannot safely click-prove destructive controls on production without a test tenant and cleanup contract — those criteria either get a tenant or stay human-gated. It cannot verify universal absence properties ("never leaks") — only falsify them when they fail during observation. It cannot verify judgement qualities without a human, and it does not pretend to. It cannot verify states the test account cannot reach (other roles, other tenants) unless each is enumerated with its own credentials in the spec's `permissions[]`. And it proves points in time — a control can die five minutes after a green run, which is what the schedule is for, and even the schedule only bounds the window.

5. Effort estimate

Assumes one engineer who knows Playwright, and your existing spec format as input.

Verification harness v1 — roughly two to three weeks: spec → manifest compiler (small; the fields already exist), census + click-proof generator with the service-worker/HAR settings above, request-ID injection and a log-lookup adapter for your backend, evidence bundle writer, and a runner exit-code contract matching your gate's style. **Signing and write-once storage — roughly one week**, using the shipped GitHub attest action and object-lock storage; early-adopter risk is schedule, not feasibility.

Ongoing cost per spec — near zero marginal tooling, real marginal authoring: the `verify` commands your gate already requires *are* the API suite; the honest recurring cost is writing them well, which the brief correctly says is real and not free. **Runtime cost:** free on CI cron; \$0–24/month hosted if you want managed scheduling and alerting.

The number to hold the approach to, from §2.5: the one measured deployment of independent mechanical verification took false success from roughly 45% of failures to roughly 3%. That is the bar, and the reason the three weeks are worth it.

Sources

Live-page proof: [Playwright network](#) · [Trace viewer](#) · [Aria snapshots](#) · [SW HAR bug #29474](#) · [keepalive #34497](#) · [HAR 1.2 spec](#) · [abandoned W3C HAR note](#) · [Heroku request IDs](#) · [Datadog synthetics ↔ APM](#) · [OTel JS browser status](#) · [Checkly pricing](#) · [Datadog RUM dead clicks](#) · [FullStory dead clicks](#) · [MS synthetic monitoring playbook](#) · [Google SRE testing](#). Tooling: [Schemathesis paper](#) · [Schemathesis repo](#) · [RESTler](#) · [Dredd \(archived\)](#) · [Microcks CNCF incubation](#) · [Optic post-mortem](#) · [Pact provider docs](#) · [Quickstrom PLDI 2022](#) · [ShardStore SOSP 2021](#) · [AWS systems correctness / PObserve](#) · [MongoDB eXtreme Modelling](#) · [Keptn archived](#) · [Tracetest](#) · [Falco](#) · [Bach & Bolton](#) · [SRE SLOs](#). False success and evidence: [Confident Closing](#) · [agent tests 2602.07900](#) · [Panickssery](#) · [Capers Jones DRE](#) · [Fagan 1976](#) · [Calikli & Bener 2013](#) · [Google flaky tests](#) · [Microsoft flakiness ICSE-SEIP 2019](#) · [in-toto test-result predicate](#) · [GitHub actions/attest](#) · [Rekor](#).